



Année Universitaire 2025–2026

SmartDiab

Application Mobile Intelligente de Gestion Alimentaire pour Diabétiques

Rapport de Projet de Fin d'Année

Réalisé par	Wissal Laaziz
Encadrant	M. Bousmah
Domaine	Santé Numérique / e-Health / IA Mobile
Technologies	Android (Java), Python FastAPI, Firebase, SQLite
Année	2025 – 2026

Dédicaces

Je dédie ce travail à mes parents, qui m'ont soutenue tout au long de mon parcours académique avec amour, patience et encouragement.

À mes frères et sœurs, pour leur présence constante et leur soutien moral précieux.

À tous mes amis et camarades de promotion, avec qui j'ai partagé les moments de travail et de réussite.

Et à toutes les personnes atteintes de diabète, pour lesquelles ce projet a été conçu avec dévouement et espoir.

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude à mon encadrant, Monsieur Bousmah, pour ses conseils avisés, sa disponibilité et son accompagnement tout au long de ce projet. Sa rigueur scientifique et son expertise ont été une source d'inspiration précieuse.

Je remercie également l'ensemble du corps enseignant de notre établissement pour la qualité de la formation dispensée et pour avoir mis à notre disposition les ressources nécessaires à la réalisation de ce travail.

Mes remerciements vont aussi à ma famille et à mes proches pour leur soutien indéfectible et leurs encouragements permanents tout au long de ce parcours.

Enfin, je remercie tous ceux qui, de près ou de loin, ont contribué à la réussite de ce projet.

Résumé

SmartDiab est une application mobile intelligente développée sous Android Studio en Java, destinée à aider les personnes diabétiques à mieux gérer leur alimentation quotidienne. Elle repose sur l'intelligence artificielle et la vision par ordinateur pour analyser les repas photographiés par l'utilisateur et lui indiquer si le plat est autorisé, à consommer avec modération ou à éviter, selon son profil diabétique (Type 1 ou Type 2).

L'application intègre une authentification sécurisée via Firebase, un questionnaire médical intelligent pour la détection du type de diabète, une analyse alimentaire en temps réel via une API dédiée, ainsi qu'un avatar animé grâce aux animations Lottie. Le backend est développé en Python avec FastAPI, et les données sont gérées via Firebase et SQLite.

Mots-clés (6 max) : Diabète, Application Mobile, Intelligence Artificielle, Analyse Alimentaire, Android, Firebase

Abstract

SmartDiab is an intelligent mobile application developed with Android Studio using Java, designed to help diabetic patients better manage their daily diet. It leverages artificial intelligence and computer vision to analyze meal photos taken by the user and determine whether a dish is allowed, to be consumed in moderation, or to be avoided, based on the user's diabetic profile (Type 1 or Type 2).

The application integrates secure authentication via Firebase, an intelligent medical questionnaire for diabetes type detection, real-time food analysis via a dedicated API, and an animated avatar using Lottie animations. The backend is developed in Python using FastAPI, with data managed through Firebase and SQLite.

Keywords (6 max): Diabetes, Mobile Application, Artificial Intelligence, Food Analysis, Android, Firebase

Liste des Abréviations

API	Application Programming Interface
IA	Intelligence Artificielle
REST	Representational State Transfer
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
UI / UX	User Interface / User Experience
ML	Machine Learning (Apprentissage Automatique)
ORM	Object-Relational Mapping
SDK	Software Development Kit
UML	Unified Modeling Language
DFD	Diagramme de Flux de Données

Liste des Figures

- Figure 1 : Architecture générale de l'application SmartDiab
- Figure 2 : Diagramme de cas d'utilisation
- Figure 3 : Diagramme d'activité — Scan d'un repas
- Figure 4 : Diagramme de classes
- Figure 5 : Interface de connexion et d'inscription
- Figure 6 : Interface du questionnaire médical
- Figure 7 : Interface de scan et d'analyse alimentaire
- Figure 8 : Résultat de l'analyse (autorisé / modération / à éviter)
- Figure 9 : Avatar animé Lottie
- Figure 10 : Diagramme de déploiement — Architecture client/serveur

Liste des Tableaux

Tableau 1 : Informations générales du projet

Tableau 2 : Technologies et outils utilisés

Tableau 3 : Description des cas d'utilisation

Tableau 4 : Règles de gestion

Tableau 5 : Comparaison des solutions existantes

Tableau 6 : Planning de réalisation (Diagramme de Gantt)

Table des Matières

Dédicaces	i
Remerciements	ii
Résumé / Abstract	iii
Liste des Abréviations	iv
Liste des Figures	v
Liste des Tableaux	vi
Introduction Générale	1
Chapitre 1 : Contexte Général du Projet	2
I. Présentation du Projet	
I-1. Problématique	
I-2. Solution Proposée	
II. Méthodologie de Travail et Planification	
Diagramme de Gantt	
Chapitre 2 : Analyse et Conception	X
I. Étude Fonctionnelle	
I-1. Analyse des Besoins Fonctionnels	
I-2. Analyse des Besoins Non Fonctionnels	
II. Étude Conceptuelle	
II-1. Règles de Gestion	
II-2. Diagrammes UML (Cas d'utilisation, Activité, Classe)	
Chapitre 3 : Étude Technique	X
I. Architecture du Projet	
II. Technologies Utilisées	
III. Environnement de Travail	
Chapitre 4 : Mise en Œuvre	X
I. Réalisation	

II. DevOps (Implémentation)

Conclusion Générale et Perspectives	X
Références	X
Annexes	X

Introduction Générale

Le diabète est l'une des maladies chroniques les plus répandues dans le monde, touchant des centaines de millions de personnes. La gestion quotidienne de cette maladie nécessite une surveillance constante de l'alimentation, une tâche souvent perçue comme complexe et contraignante pour les patients. L'inadaptation du régime alimentaire constitue l'une des principales causes de complications diabétiques graves.

Face à ce constat, les nouvelles technologies de l'information et de la communication, notamment les applications mobiles et l'intelligence artificielle, offrent des opportunités inédites pour accompagner les patients dans leur quotidien. La santé numérique, ou e-Health, représente aujourd'hui un enjeu majeur dans le secteur médical, en permettant de démocratiser l'accès aux soins et d'améliorer la qualité de vie des patients.

C'est dans ce contexte que s'inscrit notre projet SmartDiab : une application mobile intelligente développée pour Android, permettant aux personnes diabétiques d'analyser leurs repas en temps réel grâce à la vision par ordinateur et à une API d'analyse alimentaire. L'application identifie le profil diabétique de l'utilisateur et lui indique si un plat photographié est compatible avec son régime alimentaire.

Ce rapport présente l'ensemble des étapes de développement du projet, organisées en quatre chapitres principaux :

- Le Chapitre 1 présente le contexte général du projet, la problématique, la solution proposée et la planification des travaux.
- Le Chapitre 2 détaille l'analyse et la conception, incluant l'étude des besoins et les modélisations UML.
- Le Chapitre 3 traite l'étude technique : l'architecture, les technologies choisies et l'environnement de travail.
- Le Chapitre 4 est consacré à la mise en œuvre et à l'implémentation de l'application.

Nous concluons par un bilan des réalisations et les perspectives envisagées pour les évolutions futures de l'application.

Chapitre 1

Contexte Général du Projet

Introduction

Ce chapitre présente le cadre général dans lequel s'inscrit le projet SmartDiab. Nous y exposons la problématique de départ, la solution proposée, ainsi que la méthodologie de travail adoptée et la planification des différentes phases du développement.

I. Présentation du Projet

I-1. Problématique

Le diabète est une maladie chronique qui exige une gestion rigoureuse et permanente du régime alimentaire. Les personnes diabétiques doivent constamment évaluer la compatibilité des aliments qu'elles consomment avec leur état de santé. Cette contrainte, souvent mal comprise ou négligée, est l'une des principales causes de complications médicales graves telles que l'hyperglycémie, les maladies cardiovasculaires ou les troubles rénaux.

Or, les solutions numériques actuelles dédiées à la gestion du diabète sont souvent génériques, peu personnalisées et ne tiennent pas compte des différents types de diabète (Type 1 et Type 2). De plus, elles nécessitent une saisie manuelle fastidieuse des aliments consommés, ce qui limite leur adoption par les patients.

La problématique centrale de ce projet est donc la suivante : comment permettre à une personne diabétique d'évaluer rapidement et intuitivement la compatibilité d'un repas avec son profil médical, en utilisant les technologies mobiles et l'intelligence artificielle ?

I-2. Solution Proposée

SmartDiab répond à cette problématique en proposant une application mobile intelligente qui combine plusieurs technologies de pointe pour offrir une assistance alimentaire personnalisée et immédiate. La solution repose sur les principes suivants :

- Un profil médical personnalisé : lors de la première utilisation, l'utilisateur complète un questionnaire médical permettant de déterminer son type de diabète (Type 1 ou Type 2). Ces données sont sauvegardées pour éviter de répéter le questionnaire à chaque session.
- Un scan intelligent des repas : l'utilisateur photographie un plat via la caméra de son smartphone. L'image est transmise à une API d'analyse alimentaire qui identifie les aliments et évalue leur impact glycémique.

- Une recommandation claire et personnalisée : le résultat de l'analyse est présenté de manière simple (aliment autorisé, à consommer avec modération, ou à éviter), en adéquation avec le profil diabétique de l'utilisateur.
- Un avatar animé pour une meilleure expérience utilisateur : un assistant virtuel animé via Lottie accompagne l'utilisateur tout au long de son expérience.

Cette solution vise à simplifier la gestion quotidienne du diabète et à favoriser une alimentation saine et adaptée, dans le respect de la recommandation médicale fondamentale : l'application ne remplace pas la consultation médicale.

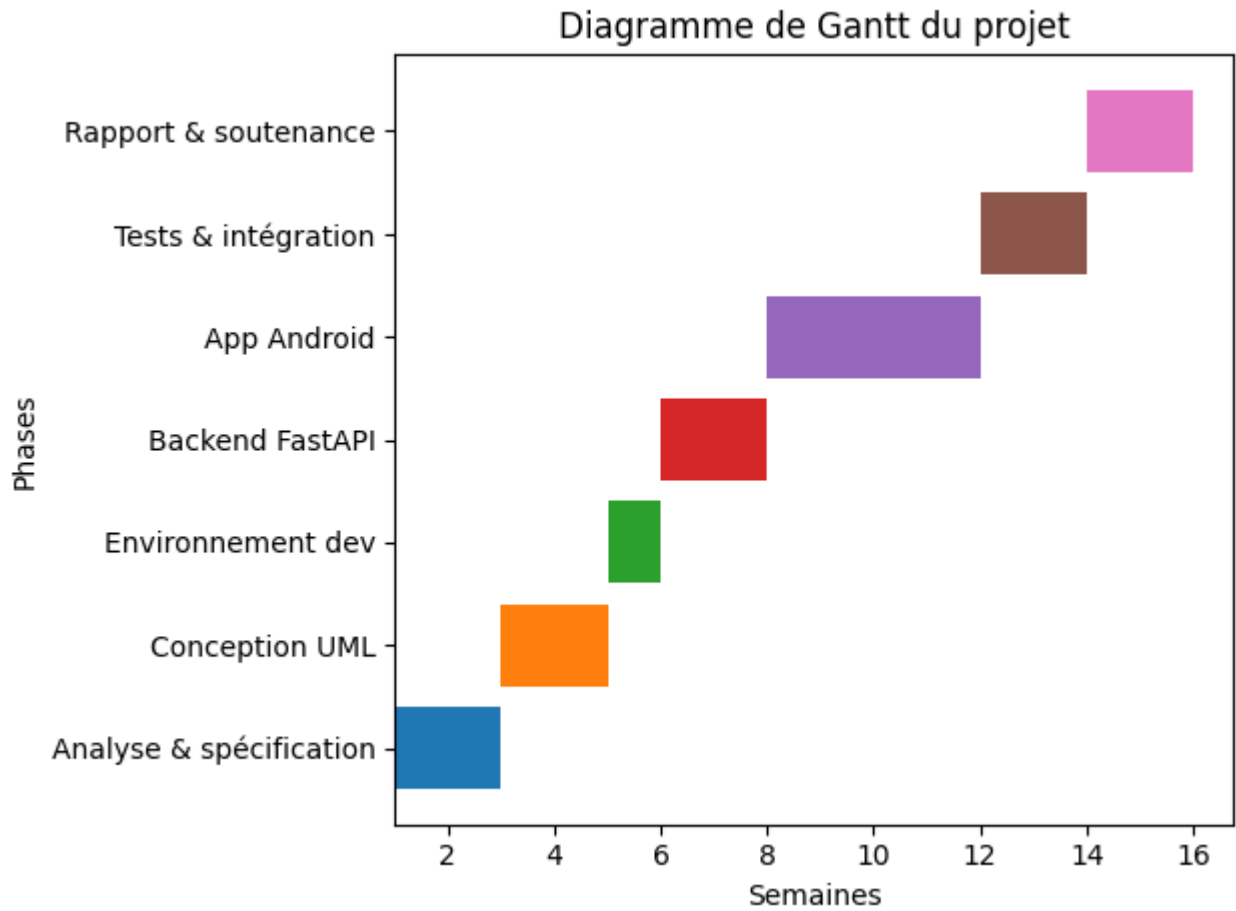
II. Méthodologie de Travail et Planification

Pour mener à bien ce projet, nous avons adopté une méthodologie de développement agile adaptée aux projets mobiles. Cette approche itérative et incrémentale nous a permis de planifier le travail en sprints successifs, d'intégrer les retours progressivement et de livrer un produit fonctionnel à chaque étape.

Les grandes phases du projet sont les suivantes :

- Phase 1 — Analyse et spécification des besoins (Semaines 1-2)
- Phase 2 — Conception et modélisation UML (Semaines 3-4)
- Phase 3 — Mise en place de l'environnement de développement (Semaine 5)
- Phase 4 — Développement du backend FastAPI (Semaines 6-7)
- Phase 5 — Développement de l'application Android (Semaines 8-11)
- Phase 6 — Intégration, tests et corrections (Semaines 12-13)
- Phase 7 — Rédaction du rapport et préparation de la soutenance (Semaines 14-15)

Le diagramme de Gantt ci-après illustre la répartition temporelle de ces différentes phases tout au long du projet.



Conclusion

Ce premier chapitre a permis de définir le cadre et les objectifs du projet SmartDiab. La problématique clairement identifiée et la solution proposée constituent la base solide sur laquelle repose l'ensemble des développements présentés dans la suite de ce rapport. Le prochain chapitre sera consacré à l'analyse détaillée des besoins et à la conception de l'application.

Chapitre 2

Analyse et Conception

Introduction

Ce chapitre est consacré à l'étude approfondie des besoins de l'application SmartDiab ainsi qu'à sa conception. Nous commençons par l'identification et l'analyse des besoins fonctionnels et non fonctionnels, puis nous procédons à la conception à travers les règles de gestion et les diagrammes UML.

I. Étude Fonctionnelle

I-1. Analyse des Besoins Fonctionnels

Les besoins fonctionnels définissent l'ensemble des fonctionnalités que l'application doit offrir pour répondre aux attentes des utilisateurs. Pour SmartDiab, nous avons identifié les fonctionnalités suivantes :

- Authentification sécurisée : l'utilisateur peut créer un compte, se connecter et se déconnecter de manière sécurisée via Firebase Authentication.
- Questionnaire médical intelligent : lors de la première connexion, un questionnaire guide l'utilisateur à travers plusieurs questions médicales pour identifier son type de diabète.
- Détection automatique du type de diabète : sur la base des réponses au questionnaire, l'application détermine automatiquement le profil diabétique (Type 1 ou Type 2).
- Scan et analyse des repas : l'utilisateur peut photographier un plat via la caméra et obtenir une analyse alimentaire immédiate.
- Affichage du résultat d'analyse : le résultat est présenté clairement (autorisé, à consommer avec modération, ou à éviter).
- Sauvegarde du profil utilisateur : les données médicales et les préférences sont stockées afin d'éviter de répéter le questionnaire.
- Historique des analyses : l'utilisateur peut consulter l'historique de ses scans alimentaires précédents.
- Avatar animé : un assistant virtuel animé accompagne l'utilisateur pour une meilleure expérience interactive.

I-2. Analyse des Besoins Non Fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité et de performance que l'application doit respecter :

- Performance : l'analyse d'un repas doit être effectuée en moins de 5 secondes dans des conditions réseau normales.

- **Sécurité** : les données médicales des utilisateurs doivent être protégées et chiffrées. L'authentification doit être robuste.
- **Disponibilité** : l'application doit fonctionner de manière stable, avec un taux de disponibilité du service backend supérieur à 99%.
- **Ergonomie et accessibilité** : l'interface doit être intuitive, claire et accessible à des utilisateurs de tout niveau technologique.
- **Compatibilité** : l'application doit être compatible avec Android 8.0 (API 26) et versions supérieures.
- **Maintenabilité** : le code doit être structuré, documenté et facilement extensible pour de futures évolutions.
- **Fiabilité de l'analyse** : le modèle d'IA doit atteindre un taux de reconnaissance alimentaire satisfaisant (> 85%).

II. Étude Conceptuelle

II-1. Règles de Gestion

Les règles de gestion définissent les contraintes métier appliquées au niveau de l'application :

RG-01	Un utilisateur doit obligatoirement créer un compte avant d'accéder aux fonctionnalités de l'application.
RG-02	Le questionnaire médical est soumis une seule fois lors de la première connexion. Les réponses sont sauvegardées en base de données.
RG-03	Le type de diabète est déterminé automatiquement par l'algorithme sur la base des réponses au questionnaire.
RG-04	Chaque analyse alimentaire est associée au profil médical de l'utilisateur connecté.
RG-05	Un plat est classifié en trois catégories : Autorisé, À consommer avec modération, À éviter.
RG-06	Les images de repas sont stockées sur Firebase Storage et associées à l'identifiant de l'utilisateur.
RG-07	L'application affiche systématiquement le message d'avertissement médical à chaque résultat d'analyse.
RG-08	Un utilisateur non connecté ne peut pas accéder aux fonctionnalités de scan ou de consultation de l'historique.

II-2. Diagrammes UML

La modélisation UML de SmartDiab comprend trois types de diagrammes principaux permettant de représenter les différents aspects du système.

Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation modélise les interactions entre les acteurs (Utilisateur et Système Backend) et les fonctionnalités principales de l'application. Les cas d'utilisation identifiés sont : S'inscrire, Se connecter, Remplir le questionnaire médical, Scanner un repas, Consulter le résultat de l'analyse, Consulter l'historique, et Se déconnecter.

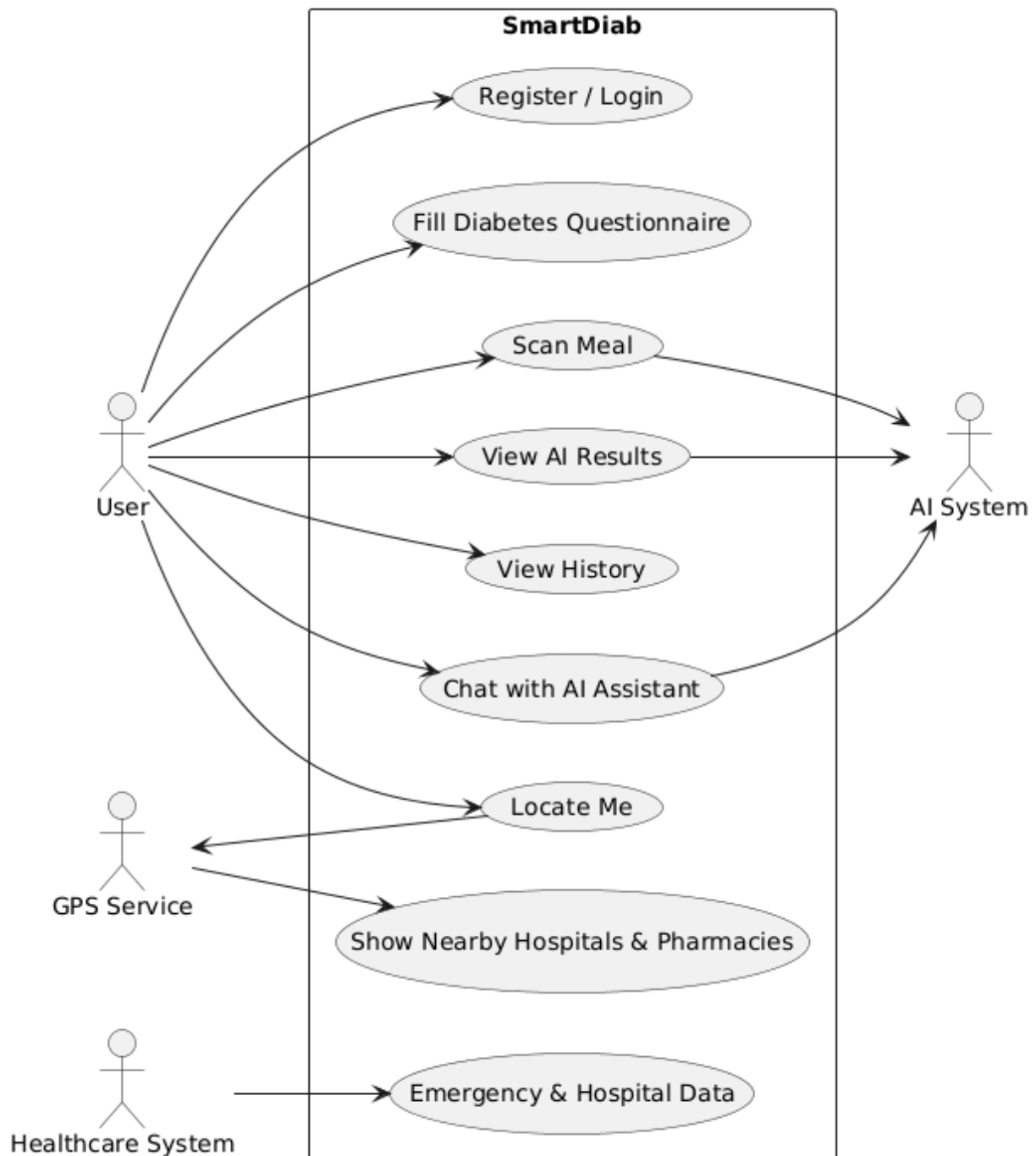


Diagramme d'Activité

Le diagramme d'activité représente le flux de traitement lors de la fonctionnalité principale de scan d'un repas. Le processus se déroule comme suit : l'utilisateur ouvre l'application, se

connecte, active la caméra, prend une photo du plat, l'image est transmise au backend via l'API REST, le modèle d'IA analyse l'image, le résultat est renvoyé et affiché à l'utilisateur avec la recommandation adaptée à son profil.

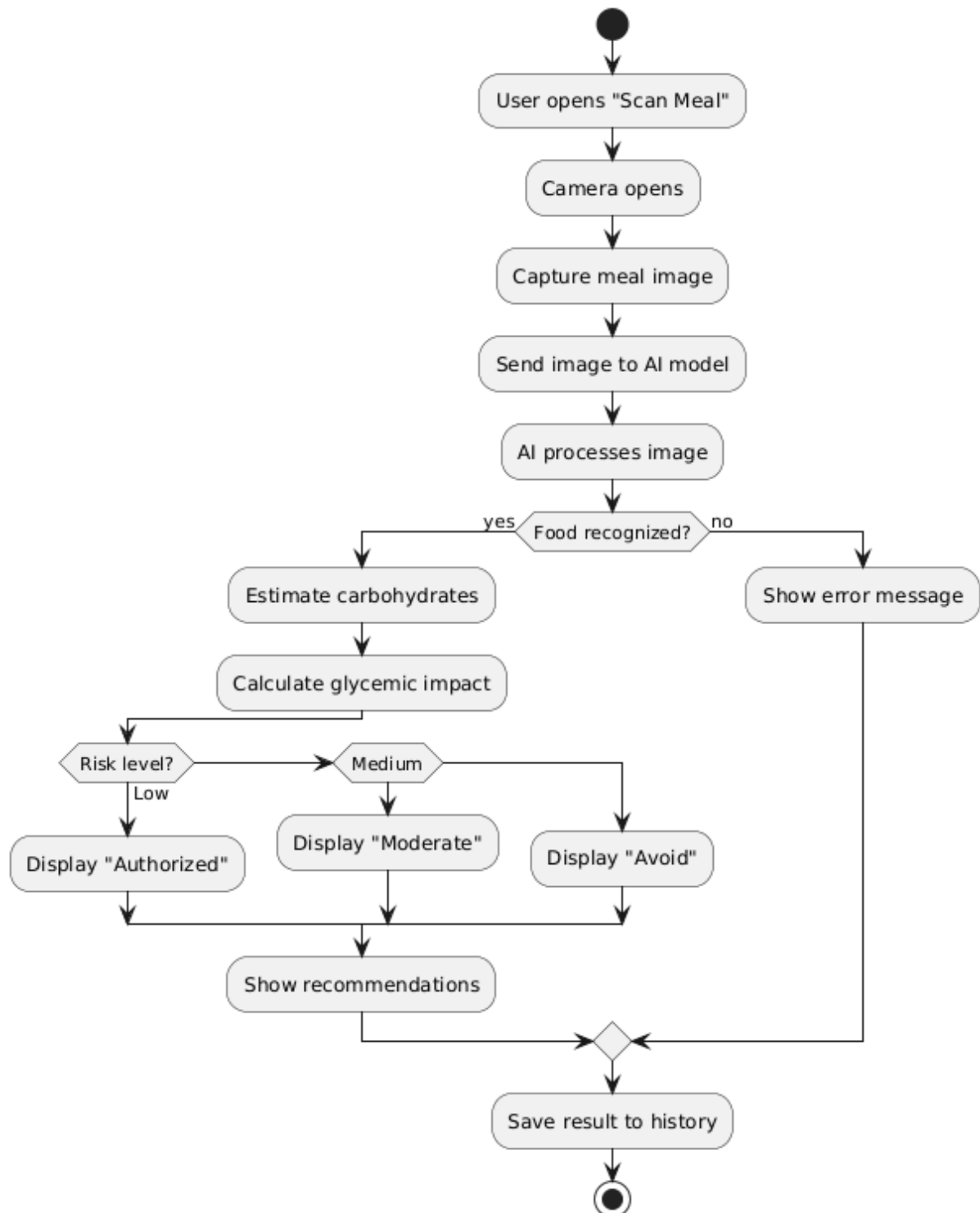
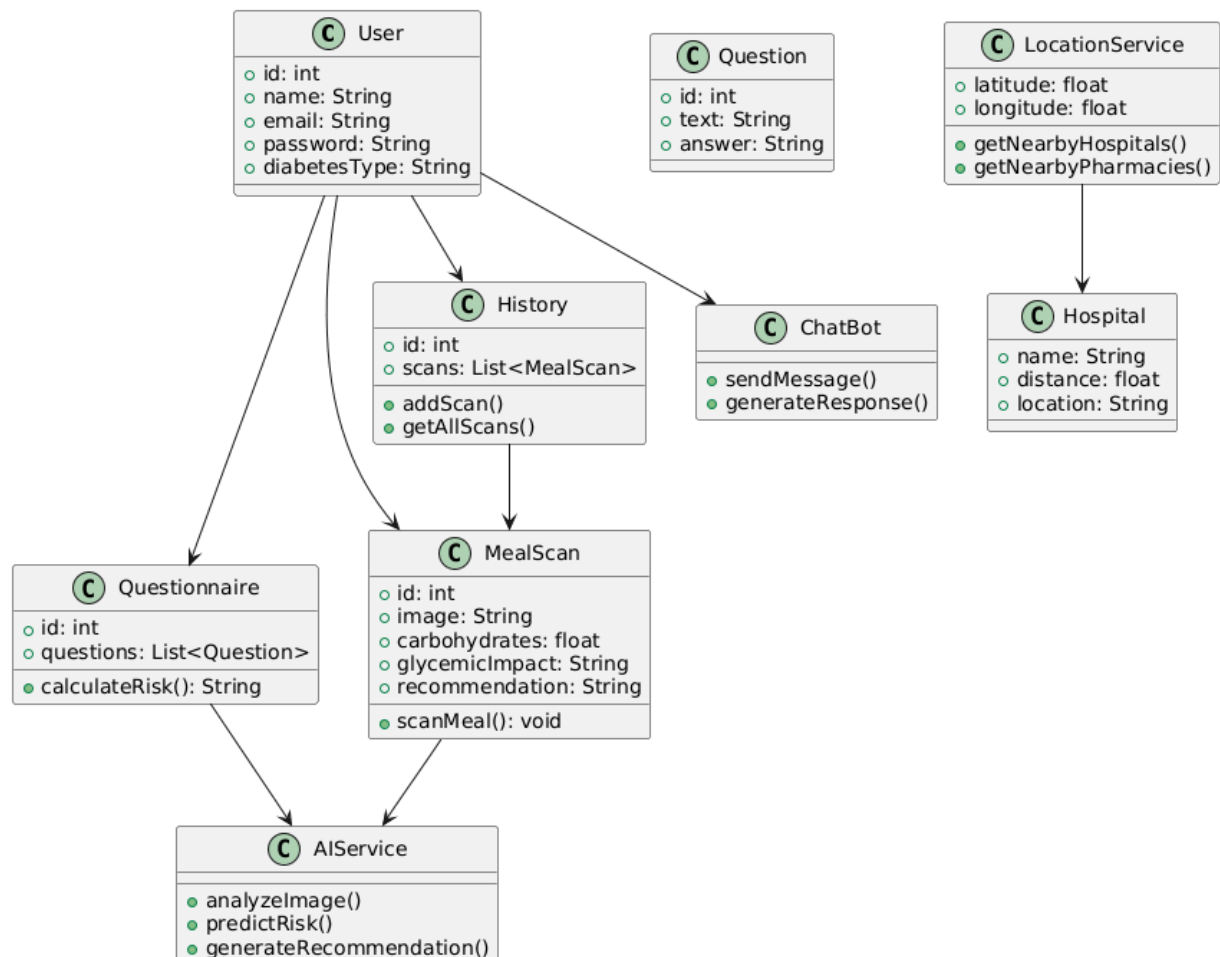


Diagramme de Classes

Le diagramme de classes modélise la structure statique de l'application, avec les entités principales suivantes : Utilisateur (identifiant, email, mot de passe, type de diabète), ProfilMédical (réponses au questionnaire, type détecté), Analyse (image, résultat, date, recommandation), et Aliment (nom, catégorie, indice glycémique, classification).



Conclusion

L'analyse fonctionnelle et la conception réalisées dans ce chapitre ont permis de définir avec précision les contours de l'application SmartDiab. Les règles de gestion établissent les contraintes métier essentielles, tandis que les diagrammes UML offrent une vision claire de l'architecture logicielle. Ces éléments constituent le socle de l'implémentation présentée dans les chapitres suivants.

Chapitre 3

Étude Technique

Introduction

Ce chapitre présente les choix techniques effectués pour le développement de SmartDiab. Nous décrivons successivement l'architecture globale du système, les technologies et outils utilisés, ainsi que l'environnement de travail mis en place.

I. Architecture du Projet

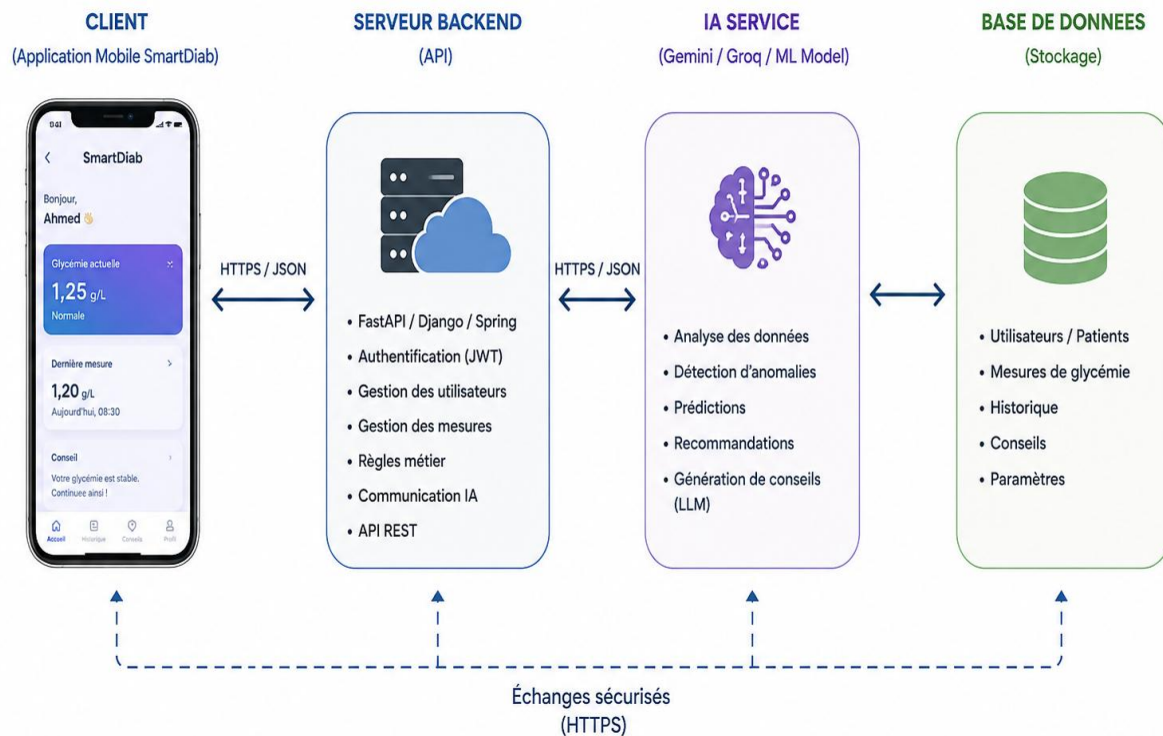
SmartDiab repose sur une architecture client-serveur moderne organisée en trois couches principales :

- La couche Présentation (Client Android) : l'application mobile développée en Java sous Android Studio constitue l'interface utilisateur. Elle gère l'affichage, les interactions utilisateur, la capture d'images et la communication avec le backend via Retrofit.
- La couche Métier (Backend FastAPI) : le serveur Python développé avec le framework FastAPI expose une API REST qui reçoit les images envoyées par le client, les traite via le modèle d'analyse alimentaire, et retourne les résultats au format JSON.
- La couche Données : la persistance des données est assurée par deux systèmes complémentaires — SQLite pour les données structurées locales (profil utilisateur, historique des analyses) et Firebase Storage pour le stockage des images capturées.

L'authentification des utilisateurs est gérée de manière transversale par Firebase Authentication, qui sécurise l'ensemble des accès à l'application.

Architecture Générale de SmartDiab

Architecture Client/Serveur



II. Technologies Utilisées

Le développement de SmartDiab a nécessité l'utilisation d'un ensemble de technologies complémentaires, choisies pour leur robustesse, leur popularité dans l'industrie et leur adéquation aux besoins du projet.

Technologie / Outil	Rôle dans le projet
Android Studio & Java	Environnement de développement et langage pour l'application mobile Android
Python & FastAPI	Développement du backend REST API, traitement des requêtes et intégration du modèle IA
Firebase Authentication	Gestion de l'authentification sécurisée des utilisateurs (inscription, connexion)
Firebase Storage	Stockage cloud des images de repas capturées par les utilisateurs
SQLite	Base de données locale pour la persistance du profil utilisateur et de l'historique

Retrofit	Bibliothèque Android pour la communication HTTP avec le backend FastAPI
API d'Analyse Alimentaire	Service externe d'identification des aliments à partir d'images (vision par ordinateur)
Lottie Animations	Bibliothèque d'animations pour l'intégration de l'avatar interactif animé
Postman	Outil de test et de documentation de l'API REST
Git & GitHub	Gestion de version et collaboration sur le code source

III. Environnement de Travail

Le développement de SmartDiab a été réalisé dans l'environnement technique suivant :

Système d'exploitation	Windows 11 (développement), Ubuntu 22.04 (serveur backend)
IDE principal	Android Studio Electric Eel / Flamingo
IDE backend	PyCharm / Visual Studio Code
Langage mobile	Java (JDK 11)
Langage backend	Python 3.10+
Version Android cible	Android 12 (API 32), minimum Android 8.0 (API 26)
Base de données	SQLite 3, Firebase Firestore
Gestion de dépendances	Gradle (Android), pip / requirements.txt (Python)
Tests	Postman (API), JUnit (Android), émulateur Android

Conclusion

Les choix technologiques effectués pour SmartDiab reflètent une architecture moderne, scalable et adaptée aux contraintes d'une application mobile de santé. L'utilisation de frameworks reconnus comme FastAPI, Firebase et Retrofit garantit la fiabilité et la maintenabilité de l'ensemble du système. Le chapitre suivant présentera la mise en œuvre concrète de ces technologies à travers les différentes étapes de l'implémentation.

Chapitre 4

Mise en Œuvre

Introduction

Ce chapitre est consacré à la réalisation concrète de l'application SmartDiab. Nous présentons successivement les étapes d'implémentation des différents modules de l'application, les interfaces réalisées, ainsi que les aspects liés au déploiement et à la mise en production.

I. Réalisation

1. Module d'Authentification

Le module d'authentification a été développé en intégrant Firebase Authentication avec le SDK Android. L'utilisateur peut créer un compte via son adresse e-mail et un mot de passe, ou se connecter à un compte existant. La gestion des erreurs (e-mail invalide, mot de passe incorrect, compte inexistant) est assurée par les callbacks Firebase, avec affichage de messages clairs à l'utilisateur.

Lors de la première connexion, l'utilisateur est automatiquement redirigé vers l'écran du questionnaire médical. Lors des connexions suivantes, les données du profil étant déjà enregistrées, il accède directement à l'interface principale de l'application.

22:16 



Bienvenue

Connectez-vous pour continuer

 Email

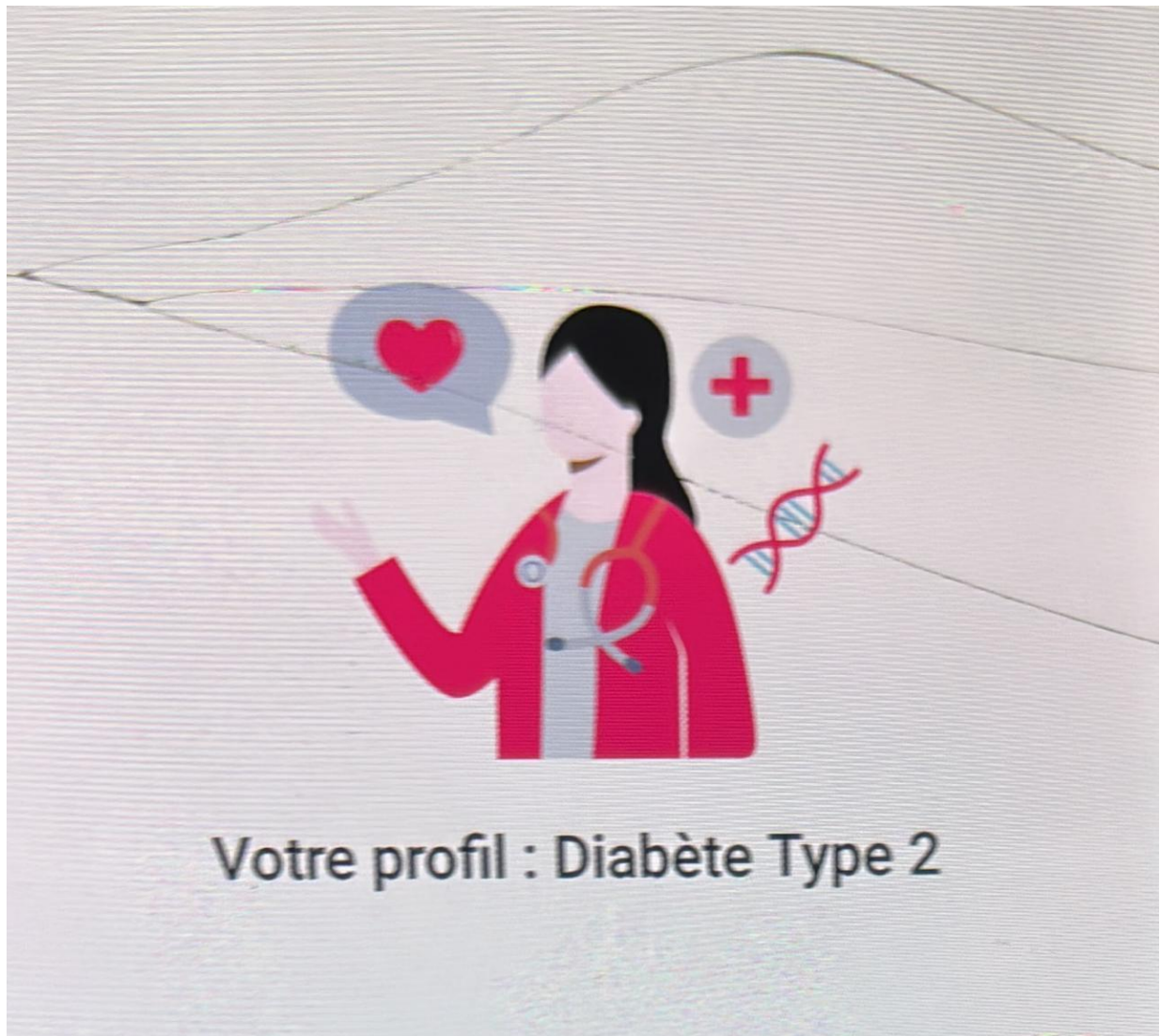
 Mot de passe



[Mot de passe oublié ?](#)

Se connecter

Nouveau ici ? Créer un compte



2. Module du Questionnaire Médical

Le questionnaire médical est présenté sous forme d'écrans successifs, chacun posant une question relative au profil diabétique de l'utilisateur (âge, antécédents familiaux, symptômes, traitement en cours, etc.). Les réponses sont analysées par un algorithme de classification qui détermine le type de diabète probable (Type 1 ou Type 2).

Les résultats sont ensuite enregistrés dans la base de données SQLite locale et associés à l'identifiant Firebase de l'utilisateur, garantissant la persistance des données entre les sessions.

22:17



Question 3 sur 10



Avez-vous une perte de poids rapide ?

Oui, absolument

Pas vraiment

3. Module de Scan et d'Analyse Alimentaire

C'est le module central de l'application. L'utilisateur active la caméra depuis l'interface principale, prend une photo du repas ou sélectionne une image depuis sa galerie. L'image est compressée et encodée en Base64, puis envoyée au backend FastAPI via une requête HTTP POST gérée par Retrofit.

Le backend reçoit l'image, l'envoie à l'API d'analyse alimentaire externe qui identifie les aliments présents dans le repas. Le modèle retourne la liste des aliments détectés avec leurs valeurs nutritionnelles et indices glycémiques. L'algorithme du backend évalue ensuite la compatibilité de ce repas avec le profil diabétique de l'utilisateur et retourne une recommandation classifiée.

4. Module de l'Avatar Animé

L'avatar interactif est intégré à l'aide de la bibliothèque Lottie pour Android. Des animations JSON prédéfinies sont chargées dynamiquement selon le contexte de l'application (accueil, analyse en cours, résultat positif, résultat négatif). Cet avatar améliore l'expérience utilisateur en rendant l'application plus engageante et intuitive.

II. DevOps & Implémentation

Déploiement du Backend

Le backend Python FastAPI est déployé sous forme de service REST exposant les endpoints nécessaires à l'application Android. Les principaux endpoints implémentés sont :

- POST /analyze — Reçoit une image encodée en Base64 et retourne l'analyse alimentaire complète.
- GET /profile/{userId} — Retourne le profil médical d'un utilisateur.
- POST /profile — Enregistre ou met à jour le profil médical d'un utilisateur.
- GET /history/{userId} — Retourne l'historique des analyses d'un utilisateur.

L'API est documentée automatiquement via Swagger UI intégré à FastAPI, ce qui facilite les tests et la maintenance.

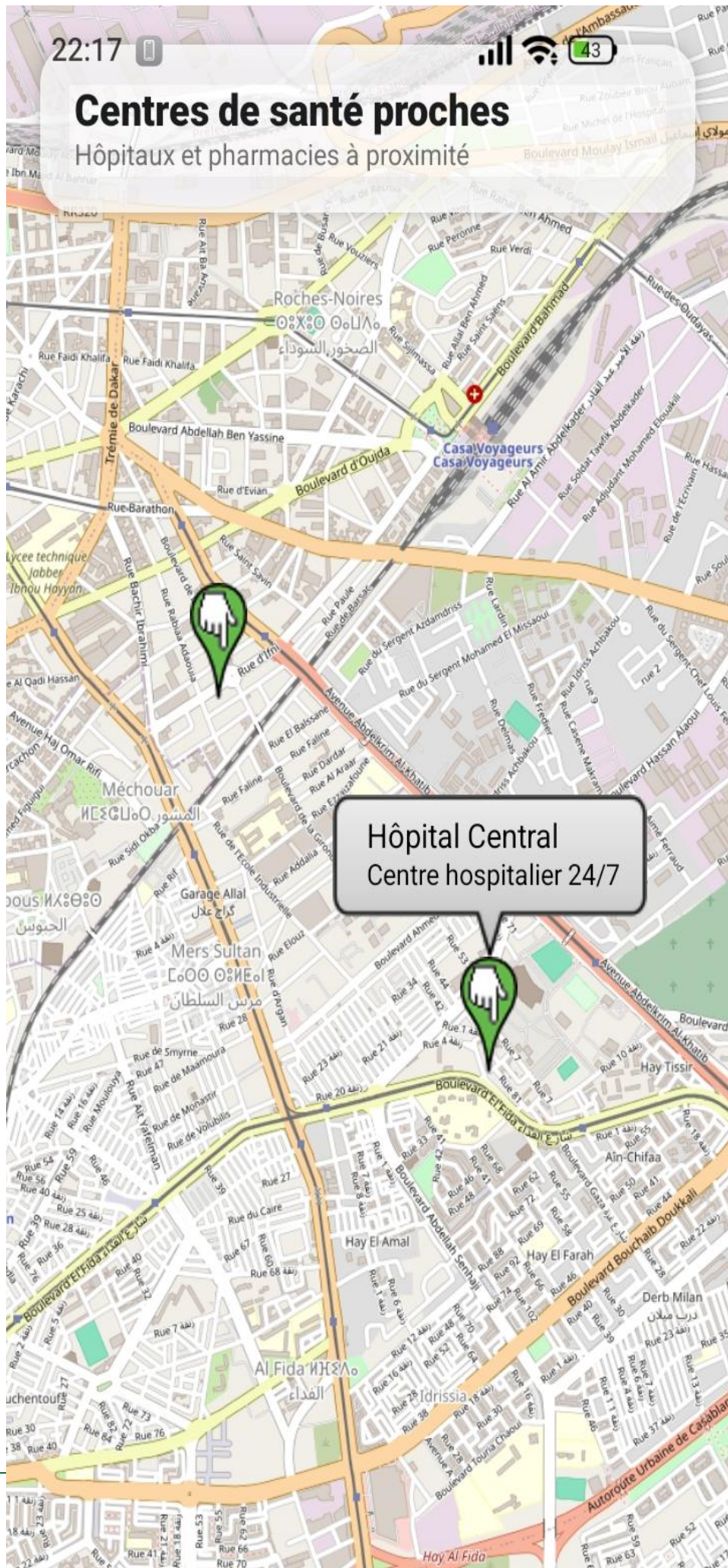
Gestion des Images avec Firebase Storage

Chaque image de repas capturée par l'utilisateur est automatiquement sauvegardée sur Firebase Storage dans un dossier structuré par identifiant utilisateur. Les URLs de téléchargement des images sont stockées en base de données pour permettre leur consultation ultérieure dans l'historique.

Tests et Validation

Les tests ont été conduits à plusieurs niveaux :

- Tests unitaires : validation des fonctions de traitement des données et de l'algorithme de classification du diabète.
- Tests d'intégration : vérification de la communication entre l'application Android et le backend FastAPI via Postman.
- Tests fonctionnels : tests des scénarios utilisateur complets sur émulateur Android et sur appareil physique.
- Tests de performance : mesure des temps de réponse de l'API d'analyse alimentaire sous différentes conditions réseau.



Conclusion

La mise en œuvre de SmartDiab a permis de concrétiser l'ensemble des fonctionnalités définies lors de la phase de conception. L'intégration des différentes technologies — Firebase, FastAPI, Retrofit, Lottie — a nécessité un effort de coordination important, mais a abouti à une application fonctionnelle, intuitive et répondant aux besoins identifiés. La conclusion générale qui suit dresse le bilan du projet et ouvre des perspectives d'évolution.

Conclusion Générale et Perspectives

Ce projet de fin d'études nous a permis de développer SmartDiab, une application mobile intelligente dédiée à la gestion alimentaire des personnes diabétiques. En combinant des technologies modernes telles qu'Android (Java), Python FastAPI, Firebase, SQLite et les APIs d'intelligence artificielle, nous avons conçu et réalisé une solution complète, fonctionnelle et orientée utilisateur.

Le projet a couvert l'intégralité du cycle de développement logiciel : de l'analyse des besoins et de la conception UML, en passant par l'implémentation des différents modules (authentification, questionnaire médical, scan alimentaire, avatar animé), jusqu'aux tests et à la validation. Les objectifs initiaux ont été atteints : l'application est capable d'identifier le profil diabétique d'un utilisateur et d'analyser ses repas en temps réel pour lui fournir des recommandations personnalisées.

Sur le plan personnel, ce projet a été une expérience extrêmement enrichissante, nous permettant de consolider nos compétences en développement mobile, en architecture logicielle, en intelligence artificielle appliquée et en gestion de projet.

Perspectives et Évolutions Futures

Plusieurs pistes d'amélioration et d'extension sont envisagées pour les versions futures de SmartDiab :

- Intégration d'un modèle de machine learning propre : développer un modèle de classification alimentaire entraîné sur un dataset spécifique aux habitudes alimentaires marocaines et méditerranéennes.
- Support multilingue : ajouter le support de l'arabe et du darija pour atteindre un public plus large.
- Module de suivi glycémique : permettre à l'utilisateur de saisir ses mesures de glycémie et d'obtenir des corrélations avec son alimentation.
- Notifications intelligentes : envoyer des rappels et des conseils nutritionnels personnalisés basés sur les habitudes alimentaires de l'utilisateur.
- Consultation médicale intégrée : permettre à l'utilisateur de partager son historique alimentaire avec son médecin via l'application.
- Version iOS : porter l'application sur la plateforme Apple pour toucher un public plus large.

- Amélioration de l'avatar IA : intégrer un chatbot conversationnel pour répondre aux questions nutritionnelles des utilisateurs.

⚠ Rappel : SmartDiab est un outil d'aide à la décision alimentaire. Il ne remplace en aucun cas l'avis et le suivi de votre médecin.

Références

- [1] Android Developers Documentation. (2024). Android Studio & Java SDK.
<https://developer.android.com>
- [2] Firebase Documentation. (2024). Firebase Authentication & Storage.
<https://firebase.google.com/docs>
- [3] FastAPI Documentation. (2024). FastAPI — Modern, fast web framework for Python.
<https://fastapi.tiangolo.com>
- [4] Airbnb / Lottie. (2024). Lottie for Android — Animations Library. <https://airbnb.io/lottie>
- [5] Square Open Source. (2024). Retrofit — A type-safe HTTP client for Android and Java.
<https://square.github.io/retrofit>
- [6] World Health Organization. (2023). Global Report on Diabetes. WHO Press.
- [7] Fédération Internationale du Diabète. (2023). IDF Diabetes Atlas, 11th Edition.
- [8] SQLite Documentation. (2024). SQLite — A self-contained SQL database engine.
<https://www.sqlite.org>

Annexes

Annexe A : Guide d'Installation et de Configuration

Installation du Backend FastAPI

1. Cloner le dépôt GitHub du projet.
2. Créer un environnement virtuel Python : `python -m venv venv`
3. Activer l'environnement et installer les dépendances : `pip install -r requirements.txt`
4. Configurer les variables d'environnement (clés API, configuration Firebase).
5. Lancer le serveur : `uvicorn main:app --reload --host 0.0.0.0 --port 8000`

Configuration de l'Application Android

1. Ouvrir le projet dans Android Studio.
2. Ajouter le fichier google-services.json (configuration Firebase) dans le dossier app/.
3. Configurer l'URL du backend dans le fichier de configuration NetworkConfig.java.
4. Compiler et déployer l'application sur un émulateur ou un appareil physique Android 8.0+.

Annexe B : Exemples de Résultats d'Analyse

Annexe C : Documentation de l'API REST

La documentation complète de l'API est accessible via l'interface Swagger UI

Endpoint	Description
POST /analyze	Analyse une image de repas et retourne la recommandation
POST /profile	Crée ou met à jour le profil médical d'un utilisateur
GET /profile/{userId}	Récupère le profil médical d'un utilisateur
GET /history/{userId}	Récupère l'historique des analyses d'un utilisateur
POST /auth/register	Enregistre un nouvel utilisateur
POST /auth/login	Authentifie un utilisateur existant